

RISC-V ISA Reference

Instruction Encoding Templates

	31	25	24	20	19	15	14	12	11	7	6	0						
R-type	funct7			rs2		rs1			funct3			rd		opcode				
I-type	imm[11:0]											rd						
S-Type	imm[11:5]			rs2								imm[4:0]						
B-type	imm[12 10:5]											imm [4:1 11]						
J-type	imm[20 10:1 11 19:12]											rd						
U-type	imm[31:12]																	

RV32I Base Integer Instruction Set

Arithmetic Instructions

	Instruction	Name	Description	Opcode	Funct 3	Funct7
R-type	add rd, rs1, rs2	<i>add</i>	$R[rd] = R[rs1] + R[rs2]$	011 0011	000	000 0000
	sub rd, rs1, rs2	<i>subtract</i>	$R[rd] = R[rs1] - R[rs2]$	011 0011	000	010 0000
	and rd, rs1, rs2	<i>bitwise and</i>	$R[rd] = R[rs1] \& R[rs2]$	011 0011	111	000 0000
	or rd, rs1, rs2	<i>bitwise or</i>	$R[rd] = R[rs1] R[rs2]$	011 0011	110	000 0000
	xor rd, rs1, rs2	<i>bitwise xor</i>	$R[rd] = R[rs1] \wedge R[rs2]$	011 0011	100	000 0000
	sll rd, rs1, rs2	<i>shift left logical</i>	$R[rd] = R[rs1] \ll R[rs2]$	011 0011	001	000 0000
	srl rd, rs1, rs2	<i>shift right logical</i>	$R[rd] = R[rs1] \gg R[rs2]$ (zero-extend)	011 0011	101	000 0000
	sra rd, rs1, rs2	<i>shift right arithmetic</i>	$R[rd] = R[rs1] \gg R[rs2]$ (sign-extend)	011 0011	101	010 0000
	slt rd, rs1, rs2	<i>set less than (signed)</i>	if ($R[rs1] < R[rs2]$) { $R[rd] = 1$; } else { $R[rd] = 0$; }	011 0011	010	000 0000
	sltu rd, rs1, rs2	<i>set less than (unsigned)</i>		011 0011	011	000 0000

	Instruction	Name	Description	Opcode	Funct 3	Funct7
I-type	<code>addi rd, rs1, imm</code>	<i>add immediate</i>	$R[rd] = R[rs1] + imm$ (sign-extend imm)	001 0011	000	n/a
	<code>andi rd, rs1, imm</code>	<i>bitwise and immediate</i>	$R[rd] = R[rs1] \& imm$ (sign-extend imm)	001 0011	111	n/a
	<code>ori rd, rs1, imm</code>	<i>bitwise or immediate</i>	$R[rd] = R[rs1] imm$ (sign-extend imm)	001 0011	110	n/a
	<code>xori rd, rs1, imm</code>	<i>bitwise xor immediate</i>	$R[rd] = R[rs1] \wedge imm$ (sign-extend imm)	001 0011	100	n/a
	<code>slti rd, rs1, imm</code>	<i>set less than immediate (signed)</i>	<pre> if (R[rs1] < imm) { R[rd] = 1; } else { R[rd] = 0; } </pre>	001 0011	010	n/a
	<code>sltiu rd, rs1, imm</code>	<i>set less than immediate (unsigned)</i>		001 0011	011	n/a
	<code>slli rd, rs1, imm</code>	<i>shift left logical immediate</i>	$R[rd] = R[rs1] \ll imm[4:0]$	001 0011	001	000 0000
	<code>srli rd, rs1, imm</code>	<i>shift right logical immediate</i>	$R[rd] = R[rs1] \gg imm[4:0]$ (zero-extend)	001 0011	101	000 0000
	<code>srai rd, rs1, imm</code>	<i>shift right arithmetic immediate</i>	$R[rd] = R[rs1] \gg imm[4:0]$ (sign-extend)	001 0011	101	010 0000

Memory Instructions

	Instruction	Name	Description	Opcode	Funct 3
I-type	lb rd, imm(rs1)	load byte	$R[rd] = M[R[rs1] + imm][7:0]$ (sign-extend)	000 0011	000
	lbu rd, imm(rs1)	load byte (unsigned)	$R[rd] = M[R[rs1] + imm][7:0]$ (zero-extend)	000 0011	100
	lh rd, imm(rs1)	load half-word	$R[rd] = M[R[rs1] + imm][15:0]$ (sign-extend)	000 0011	001
	lhu rd, imm(rs1)	load half-word (unsigned)	$R[rd] = M[R[rs1] + imm][15:0]$ (zero-extend)	000 0011	101
	lw rd, imm(rs1)	load word	$R[rd] = M[R[rs1] + imm][31:0]$ (sign-extend in RV64I)	000 0011	010
S-Type	sb rs2, imm(rs1)	store byte	$M[R[rs1] + imm][7:0] = R[rs2][7:0]$	010 0011	000
	sh rs2, imm(rs1)	store half-word	$M[R[rs1] + imm][15:0] = R[rs2][15:0]$	010 0011	001
	sw rs2, imm(rs1)	store word	$M[R[rs1] + imm][31:0] = R[rs2][31:0]$	010 0011	010

Control Instructions

	Instruction	Name	Description	Opcode	Funct 3
B-type	beq rs1, rs2, label	branch if equal	if ($R[rs1] == R[rs2]$) { PC = PC + offset }	110 0011	000
	bne rs1, rs2, label	branch if not equal	if ($R[rs1] != R[rs2]$) { PC = PC + offset }	110 0011	001
	blt rs1, rs2, label	branch if less than (signed)	if ($R[rs1] < R[rs2]$) { PC = PC + offset }	110 0011	100
	bltu rs1, rs2, label	branch if less than (unsigned)		110 0011	110
	bge rs1, rs2, label	branch if greater or equal (signed)	if ($R[rs1] >= R[rs2]$) { PC = PC + offset }	110 0011	101
	bgeu rs1, rs2, label	branch if greater or equal (unsigned)		110 0011	111

	Instruction	Name	Description	Opcode	Funct 3
J-type	<code>jal rd, label</code>	<i>jump and link</i>	$R[rd] = PC + 4$ $PC = PC + \text{offset}$	110 1111	<i>n/a</i>
I-type	<code>jalr rd, rs1, label</code>	<i>jump and link register</i>	$R[rd] = PC + 4$ $PC = R[rs1] + \text{imm}$	110 0111	000

Other Instructions

	Instruction	Name	Description	Opcode	Funct 3
U-type	<code>auipc rd, immu</code>	<i>add upper immediate to PC</i>	$R[rd] = PC + \text{imm} \ll 12$ (sign-extend shifted immediate)	011 0111	<i>n/a</i>
	<code>lui rd, immu</code>	<i>load upper immediate</i>	$R[rd] = \text{imm} \ll 12$ (sign-extend shifted immediate)	011 0111	<i>n/a</i>
I-type	<code>ebreak</code>	<i>environment BREAK</i>	<i>asks the debugger to do something (imm=0)</i>	111 0011	000
	<code>ecall</code>	<i>environment ECALL</i>	<i>asks the OS to do something (imm=1)</i>	111 0011	000

RV32A/RV64A Standard Extension for Atomic Instructions (partial list)

Info

The following table lists the load-reserved and store-conditional instructions that are available for RV32 and RV64. For documentation of the *atomic memory operations* (`amo`) see the [RISC-V Instruction Set Manual, Vol. 1, Chapter 9.4](#)

	Instruction	Name	Opcode	Funct3	Funct7
R-type	<code>lr.w rd, rs1, rs2</code>	<i>load reserved word</i>	010 1111	010	00010 <i>aq</i> <i>rl</i>
	<code>lr.d rd, rs1, rs2</code>	<i>load reserved double-word (RV64 only)</i>	010 1111	011	00010 <i>aq</i> <i>rl</i>
	<code>sc.w rd, rs1, rs2</code>	<i>store conditional word</i>	010 1111	010	00011 <i>aq</i> <i>rl</i>
	<code>sc.d rd, rs1, rs2</code>	<i>store conditional double-word (RV64 only)</i>	010 1111	011	00011 <i>aq</i> <i>rl</i>

RV64I Base Integer Instruction Set (in addition to RV32I)

Info

The 15 instructions in the RV64I set build on the RV32I variants by providing support for 64-bit load/store operations and *extensions* of operations on 32-bit values and results. There is a more detailed discussion of the problems that arise in extension in the [class notes](#).

	Instruction	Name	Description	Opcode	Funct3	Funct7
R-type	<code>addw rd, rs1, rs2</code>	<i>add word</i>	$R[rd] = (R[rs1][31:0] + R[rs2][31:0])[31:0]$ (<i>sign-extend</i>)	011 1011	000	000 0000
	<code>subw rd, rs1, rs2</code>	<i>subtract word</i>	$R[rd] = (R[rs1][31:0] - R[rs2][31:0])[31:0]$ (<i>sign-extend</i>)	011 1011	000	010 0000
	<code>sllw rd, rs1, rs2</code>	<i>shift left logical word</i>	$R[rd] = (R[rs1][31:0] \ll R[rs2])[31:0]$ (<i>sign-extend</i>)	011 1011	001	000 0000
	<code>srlw rd, rs1, rs2</code>	<i>shift right logical word</i>	$R[rd] = (R[rs1][31:0] \gg R[rs2])[31:0]$ (<i>zero-extend</i>)	011 1011	101	000 0000
	<code>sraw rd, rs1, rs2</code>	<i>shift right arithmetic word</i>	$R[rd] = (R[rs1][31:0] \ggg R[rs2])[31:0]$ (<i>sign-extend</i>)	011 1011	101	010 0000

	Instruction	Name	Description	Opcod e	Funct 3	Funct 7
I-type	<code>slli rd, rs1, imm</code>	<i>shift left logical immediate</i>	$R[rd] = R[rs1] \ll imm[5:0]$	001 0011	001	<i>n/a</i>
	<code>srli rd, rs1, imm</code>	<i>shift right logical immediate</i>	$R[rd] = R[rs1] \gg imm[5:0]$ (zero-extend)	001 0011	101	<i>n/a</i>
	<code>srai rd, rs1, imm</code>	<i>shift right arithmetic immediate</i>	$R[rd] = R[rs1] \gg imm[5:0]$ (sign-extend)	001 0011	101	<i>n/a</i>
	<code>addiw rd, rs1, imm</code>	<i>add immediate word</i>	$R[rd] = (R[rs1] + imm)[31:0]$ (sign-extend imm and result)	001 1011	000	<i>n/a</i>
	<code>slliw rd, rs1, imm</code>	<i>shift left logical immediate word</i>	$R[rd] = (R[rs1][31:0] \ll imm)[31:0]$ (sign-extend)	001 1011	001	000 0000
	<code>srliw rd, rs1, imm</code>	<i>shift right logical immediate word</i>	$R[rd] = (R[rs1][31:0] \gg imm)[31:0]$ (zero-extend)	001 1011	101	000 0000
	<code>sraiw rd, rs1, imm</code>	<i>shift right arithmetic immediate word</i>	$R[rd] = (R[rs1][31:0] \gg imm)[31:0]$ (sign-extend)	001 1011	101	010 0000
Load/Store (I-type/S-type)	<code>lwu rd, imm(rs1)</code>	<i>load word (unsigned)</i>	$R[rd] = M[R[rs1] + imm][31:0]$ (zero-extend)	000 0011	110	<i>n/a</i>
	<code>ld rd, imm(rs1)</code>	<i>load double word</i>	$R[rd] = M[R[rs1] + imm]$	000 0011	011	<i>n/a</i>
	<code>sd rs2, imm(rs1)</code>	<i>store double word</i>	$M[R[rs1] + imm] = R[rs2]$	010 0011	011	<i>n/a</i>

Standard Extension for Integer Multiplication and Division (RV32M and RV64M)

Let x_{LEN} = the width of an integer register in bits.

	Instruction	Name	Description	Opcode	Funct 3	Funct7
RV32M (R-type)	<code>mul rd, rs1, rs2</code>	<i>multiply</i>	$R[rd] = R[rs1] * R[rs2]$	011 0011	000	000 0001
	<code>mulh rd, rs1, rs2</code>	<i>multiply upper half</i>	$R[rd] = (R[rs1] * R[rs2])$ [(2×XLEN-1):XLEN] (R[rs1] and R[rs2] signed)	011 0011	001	000 0001
	<code>mulhu rd, rs1, rs2</code>	<i>multiply upper half unsigned</i>	$R[rd] = (R[rs1] * R[rs2])$ [(2×XLEN-1):XLEN] (R[rs1] and R[rs2] unsigned)	011 0011	011	000 0001
	<code>mulhsu rd, rs1, rs2</code>	<i>multiply upper half signed × unsigned</i>	$R[rd] = (R[rs1] * R[rs2])$ [(2×XLEN-1):XLEN] (signed R[rs1] and unsigned R[rs2])	011 0011	010	000 0001
	<code>div rd, rs1, rs2</code>	<i>quotient (signed)</i>	$R[rd] = R[rs2] == 0 ? -1 :$ $R[rs1] / R[rs2]$ (R[rs1] and R[rs2] signed, the quotient of a signed division with overflow is equal to the dividend)	011 0011	100	000 0001
	<code>divu rd, rs1, rs2</code>	<i>quotient (unsigned)</i>	$R[rd] = R[rs2] == 0 ? -1 :$ $R[rs1] / R[rs2]$ (R[rs1] and R[rs2] unsigned)	011 0011	101	000 0001
	<code>rem rd, rs1, rs2</code>	<i>remainder (signed)</i>	$R[rd] = R[rs2] == 0 ? R[rs1] :$ $R[rs1] \% R[rs2]$ (R[rs1] and R[rs2] signed)	011 0011	110	000 0001
	<code>remu rd, rs1, rs2</code>	<i>remainder (unsigned)</i>	$R[rd] = R[rs2] == 0 ? R[rs1] :$ $R[rs1] \% R[rs2]$ (R[rs1] and R[rs2] unsigned)	011 0011	111	000 0001

	Instruction	Name	Description	Opcode	Funct 3	Funct7
RV64M (R-type)	<code>mulw rd, rs1, rs2</code>	<i>multiply word</i>	$R[rd] = R[rs1][31:0] * R[rs2][31:0]$ (sign-extend)	011 1011	000	000 0001
	<code>divw rd, rs1, rs2</code>	<i>quotient (signed) word</i>	$R[rd] = R[rs2][31:0] == 0 ? -1 : R[rs1][31:0] / R[rs2][31:0]$ ($R[rs1][31:0]$ and $R[rs2][31:0]$ signed, result is sign-extended, the quotient of a signed division with overflow is equal to the dividend)	011 1011	100	000 0001
	<code>divuw rd, rs1, rs2</code>	<i>quotient (unsigned) word</i>	$R[rd] = R[rs2][31:0] == 0 ? -1 : R[rs1][31:0] / R[rs2][31:0]$ ($R[rs1][31:0]$ and $R[rs2][31:0]$ unsigned, result is sign-extended)	011 1011	101	000 0001
	<code>remw rd, rs1, rs2</code>	<i>remainder (signed) word</i>	$R[rd] = R[rs2][31:0] == 0 ? R[rs1][31:0] : R[rs1][31:0] \% R[rs2][31:0]$ ($R[rs1][31:0]$ and $R[rs2][31:0]$ signed, result is sign-extended)	011 1011	110	000 0001
	<code>remuw rd, rs1, rs2</code>	<i>remainder (unsigned) word</i>	$R[rd] = R[rs2][31:0] == 0 ? R[rs1][31:0] : R[rs1][31:0] \% R[rs2][31:0]$ ($R[rs1][31:0]$ and $R[rs2][31:0]$ unsigned, result is sign-extended)	011 1011	111	000 0001